Polymarket Arbitrage Bible: The Real Gap is in the Mathematical Infrastructure

Mar 11, 2026 13:34:42

Share to

Original Title: The Math Needed for Trading on Polymarket (Complete Roadmap)

Original Author: Roan, Crypto Analyst

Original Compiler: MrRyanChi

In the process of founding @insidersdotbot, I have had in-depth exchanges with many high-frequency trading teams and arbitrage teams, among which the biggest demand is how to develop arbitrage strategies.

Our users, friends, and partners are all exploring the complex and multi-dimensional trading route of arbitrage on Polymarket. If you are an active Twitter user, I believe you have also come across tweets like "I made money on the prediction market through XX arbitrage strategy."

However, most articles overly simplify the underlying logic of arbitrage, turning it into a trading model of "I can do it too" or "using Clawdbot can solve it," without explaining in detail how to systematically understand and develop one's own arbitrage system.

If you want to understand how arbitrage tools on Polymarket make money, this article is the most complete interpretation I have seen so far.

Since the original English text contains many overly technical parts that require further research, I have restructured and supplemented it to make it easier for everyone to understand all the key points without stopping to look up information.

Arbitrage on Polymarket is Not Just a Simple Math Problem

You see a market on Polymarket:

YES price $0.62, NO price $0.33.

You think: 0.62 + 0.33 = 0.95, less than $1, there is arbitrage space! Buy YES and NO simultaneously, spend $0.95, and regardless of the outcome, you can get back $1.00, netting $0.05.

You are correct.

But the problem is—while you are still manually calculating this addition, the quantitative system is doing something completely different.

It is simultaneously scanning 17,218 conditions, spanning 2^63 possible outcome combinations, finding all pricing contradictions within milliseconds. By the time you place your two orders, the price difference has already disappeared. The system has already found the same loophole in dozens of related markets, calculated the optimal position size considering order book depth and fees, executed all trades in parallel, and then directed the funds to the next opportunity.[1]

The gap is not just speed. It is mathematical infrastructure.

Chapter 1: Why "Addition" is Not Enough—The Marginal Polytope Problem

Single Market Fallacy

Let’s look at a simple example.

Market A: "Will Trump win the election in Pennsylvania?"

YES price $0.48, NO price $0.52. Together they add up to exactly $1.00.

Looks perfect, no arbitrage space, right?

Wrong.

Add a Market, and the Problem Arises

Now consider Market B: "Will the Republican Party exceed their opponent by more than 5 percentage points in Pennsylvania?"

YES price $0.32, NO price $0.68. Together they also add up to $1.00.

Both markets are "normal" on their own. But there is a logical dependency here:

The U.S. presidential election does not count votes nationwide at once, but rather by state. Each state is an independent "battleground"; whoever receives more votes in that state wins all of its electoral votes (winner-takes-all). Trump is the Republican candidate. Therefore, "the Republican Party wins in Pennsylvania" and "Trump wins in Pennsylvania"—are the same thing. If the Republican Party wins by more than 5 percentage points, it not only means Trump won Pennsylvania but won it decisively.

In other words, the YES in Market B (Republican landslide) is a subset of the YES in Market A (Trump wins)—a landslide necessarily means a win, but a win does not necessarily mean a landslide.

And this logical dependency creates arbitrage opportunities.

It's like you are betting on two things—"Will it rain tomorrow?" and "Will there be a thunderstorm tomorrow?"

If there is a thunderstorm, it must be raining (a thunderstorm is a subset of rain). Therefore, the price of "thunderstorm YES" cannot be higher than the price of "rain YES." If market pricing violates this logic, you can buy low and sell high simultaneously, earning "risk-free profit," which is arbitrage.

Exponential Explosion: Why Brute Force Search Doesn't Work

For any market with n conditions, there are theoretically 2^n possible price combinations.

Sounds manageable? Let's look at a real case.

2010 NCAA Championship Market [2]: 63 games, each with two outcomes (win/lose). The number of possible outcome combinations is 2^{63} = 9,223,372,036,854,775,808—over 9 quintillion. There are over 5,000 markets in the system.

How big is the number 2^{63}? If you check 1 billion combinations per second, it would take about **292 years** to check them all. This is why "brute force search" is completely impractical here. Checking each combination one by one? Computationally impossible.

Now consider the 2024 U.S. election. Research teams identified 1,576 pairs of markets that may have dependencies. If each pair has 10 conditions, then each pair needs to check 2^{20} = 1,048,576 combinations. Multiply that by 1,576 pairs. By the time your laptop finishes calculating, the election results will already be out.

Integer Programming: Using Constraints Instead of Enumeration

The solution for quantitative systems is not "enumerating faster," but rather not enumerating at all.

They use Integer Programming to describe "which outcomes are valid."

Let’s look at a real example. The market for the Duke vs. Cornell game: each team has 7 outcomes (0 to 6 wins), totaling 14 conditions, 2^{14} = 16,384 possible combinations.

But there is one constraint: they cannot both win more than 5 games, because then they would meet in the semifinals (only one can advance).

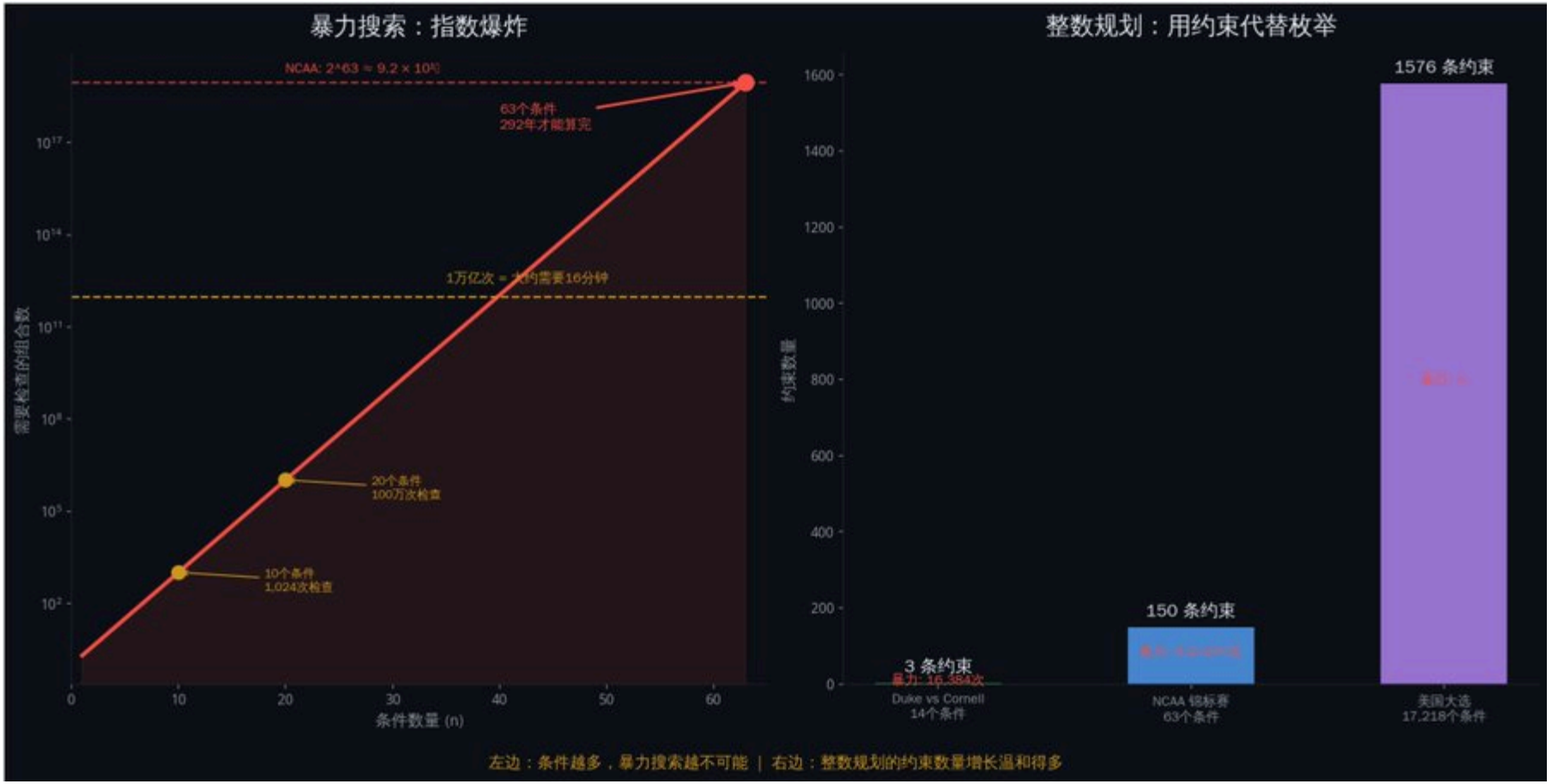
How does integer programming handle this? Three constraints are sufficient:

Constraint 1: Exactly one of Duke's 7 outcomes is true (Duke can only have one final win count).

Constraint 2: Exactly one of Cornell's 7 outcomes is true.

Constraint 3: Duke wins 5 + Duke wins 6 + Cornell wins 5 + Cornell wins 6 is less than or equal to 1 (they cannot both win that many).

Three linear constraints replace 16,384 brute force checks.



Brute Force Search vs. Integer Programming

In other words, brute force search is like reading every word in the dictionary to find one word. Integer programming is like flipping directly to the page that starts with that letter. You don't need to check all possibilities; you just need to describe "what a valid answer looks like," and then let the algorithm find the pricing that violates the rules.

Real Data: 41% of Markets Have Arbitrage [2]

The original text mentions that the research team analyzed data from April 2024 to April 2025:

Checked 17,218 conditions

Among them, 7,051 conditions had single market arbitrage (41%)

Median pricing deviation: $0.60 (should be $1.00)

13 pairs of confirmed cross-market arbitrage opportunities

A median deviation of $0.60 means the market frequently deviates by 40%. This is not "close to efficient"; this is "massively exploitable."

Why "Straight-Line Distance" Doesn't Work

The most intuitive idea is: find the "valid price" closest to the current price, and then trade the difference.

In mathematical terms, this means minimizing the Euclidean distance: $||\mu - \theta||^2$

But there is a fatal flaw: it treats all price movements as equal.

An increase from $0.50 to $0.60 and an increase from $0.05 to $0.15 are both a 10-cent increase. But their informational content is completely different.

Why? Because prices represent implied probabilities. A change from 50% to 60% is a mild adjustment in perspective. A change from 5% to 15% is a massive shift in belief—an event that was almost impossible suddenly becomes "somewhat possible."

Imagine you are weighing yourself. If you go from 70 kg to 80 kg, you might say, "I gained a little weight." But if you go from 30 kg to 40 kg (if you are an adult), that's "from near death to severely malnourished." The same 10 kg change has completely different meanings. Prices are the same—movements closer to 0 or 1 carry more information.

Bregman Divergence: The Correct "Distance"

Polymarket's market makers use LMSR (Logarithmic Market Scoring Rule) [4], where prices essentially represent probability distributions.

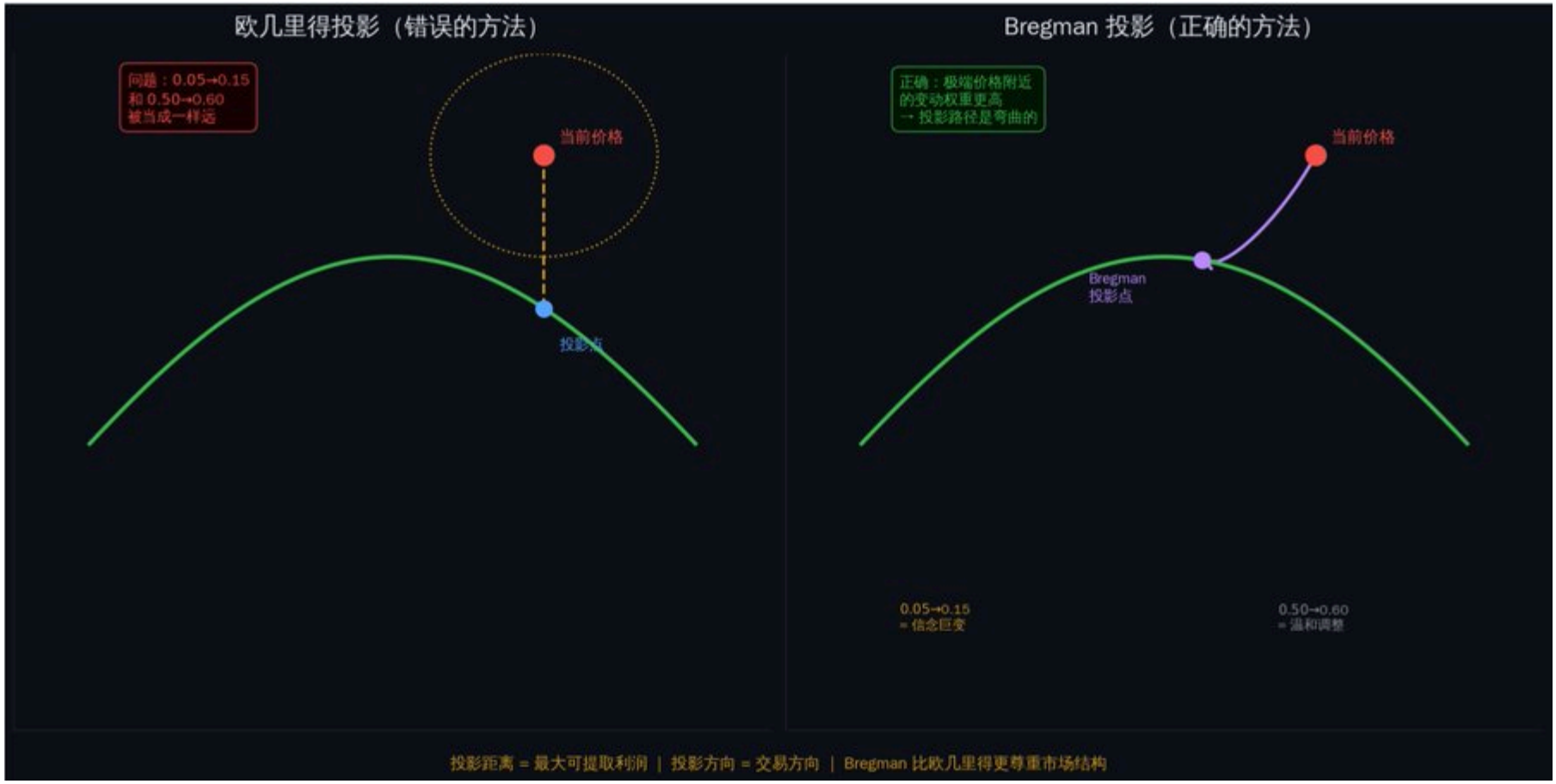
In this structure, the correct distance measure is not Euclidean distance, but **Bregman Divergence**.[5]

For LMSR, Bregman divergence becomes KL divergence (Kullback-Leibler divergence)[6]—a measure of the "information-theoretic distance" between two probability distributions.

You don’t need to remember the formula. You just need to understand one thing:

KL divergence automatically gives higher weight to "movements near extreme prices." A movement from $0.05 to $0.15 is "further" under KL divergence than a movement from $0.50 to $0.60. This aligns perfectly with our intuition—movements in extreme prices imply larger informational shocks.

A good example is when Axiom overtook Meteora at the last moment in @zachxbt's prediction market, also characterized by extreme price movements.



Bregman Projection vs. Euclidean Projection

Arbitrage Profit = Distance of Bregman Projection

This is one of the core conclusions referenced by the original author throughout the paper:

The maximum guaranteed profit that any trade can achieve equals the distance from the current market state to the arbitrage-free space in Bregman projection.

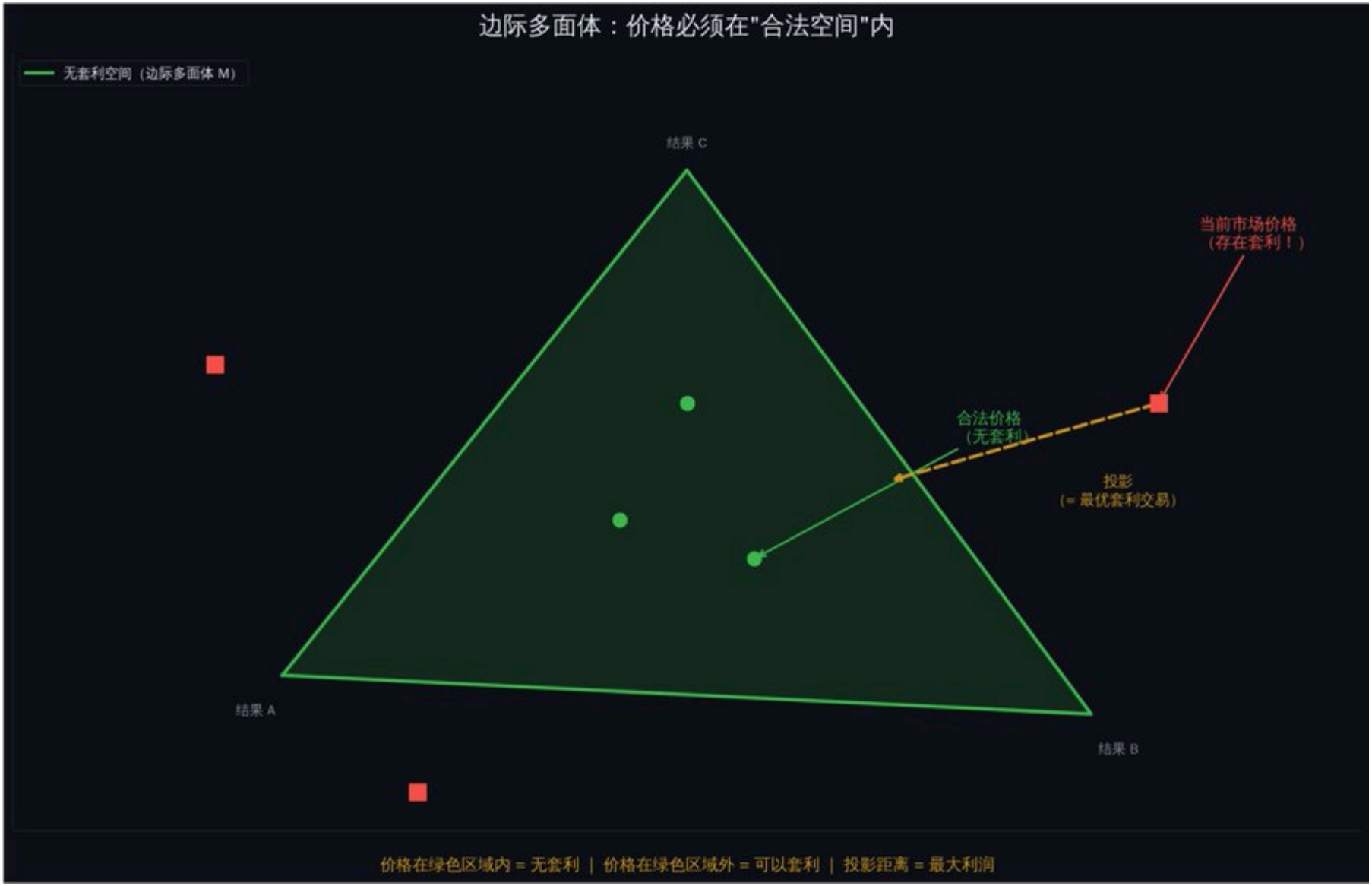
In other words: the further the market price deviates from the "valid space," the more money can be made. And Bregman projection will tell you:

What to buy and sell (the projection direction indicates the trading direction)

How much to buy and sell (considering order book depth)

How much can be earned (the projection distance is the maximum profit)

The top arbitrator earned $2,009,631.76 in one year.[2] His strategy was to solve this optimization problem faster and more accurately than anyone else.



Marginal Polytope and Arbitrage

To put it metaphorically, imagine you are standing on a mountain, and at the foot of the mountain is a river (the arbitrage-free space). Your current position (the current market price) is a certain distance from the river.

Bregman projection helps you find "the shortest path from your position to the riverbank"—but not a straight line distance, rather the shortest path considering the terrain (market structure). The length of this path is the maximum profit you can

earn.

Chapter 3: Frank-Wolfe Algorithm—Turning Theory into Executable Code

Now you know: to calculate optimal arbitrage, you need to perform Bregman projection.

But the problem is—directly calculating Bregman projection is not feasible.

Why? Because the arbitrage-free space (Marginal Polytope M) has exponentially many vertices. Standard convex optimization methods require access to the complete set of constraints, which means enumerating every valid outcome. As we just discussed, this is impossible in scaled scenarios.

Core Idea of Frank-Wolfe

The genius of the **Frank-Wolfe algorithm** [7] is that it does not attempt to solve the entire problem at once, but rather approaches the answer step by step.

It works like this:

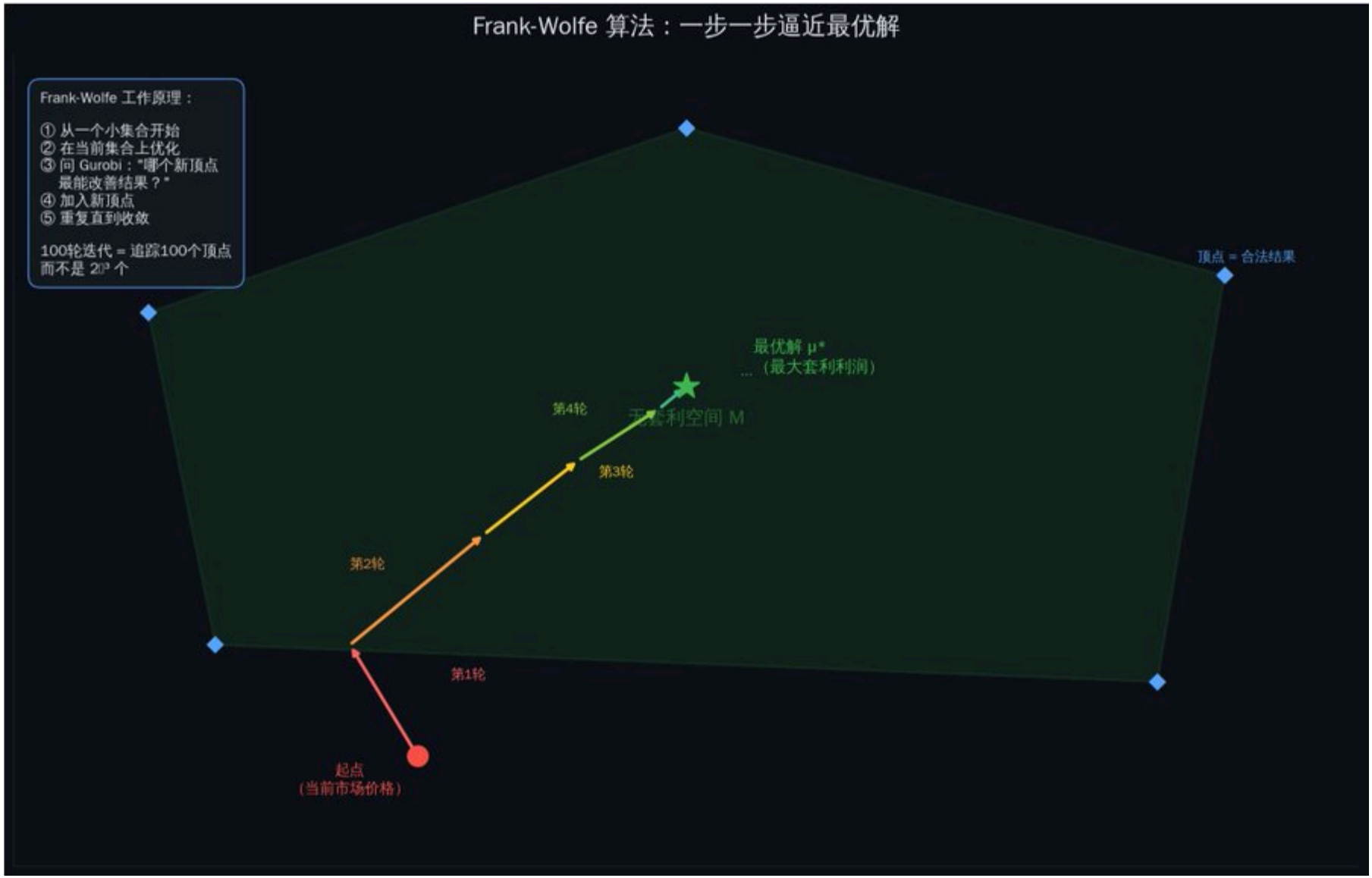
Step 1: Start from a small known set of valid outcomes.

Step 2: Optimize within this small set to find the current optimal solution.

Step 3: Use integer programming to find a new valid outcome and add it to the set.

Step 4: Check if it is close enough to the optimal solution. If not, return to Step 2.

In each iteration, the set only adds one vertex. Even after 100 iterations, you only need to track 100 vertices—rather than 2^{63} of them.



Frank-Wolfe Iteration Process

Imagine you are looking for an exit in a huge maze.

The brute force method is to walk every path. Frank-Wolfe's method is: first, walk down a random path, then at each fork, ask a "guide" (integer programming solver): "From here, which direction is most likely to lead to the exit?" Then take a step in that direction. You don’t need to explore the entire maze; you just need to make the right choice at each key node.

Integer Programming Solver: The "Guide" for Each Step

Each iteration of Frank-Wolfe requires solving an integer linear programming problem. This is theoretically NP-hard (meaning "there is no known fast general algorithm").

But modern solvers, like Gurobi[8], can solve well-structured problems efficiently.

The research team used Gurobi 5.5. Actual solving times:

Early iterations (few games finished): under 1 second

Mid-stage (30-40 games finished): 10-30 seconds

Late-stage (50+ games finished): under 5 seconds

Why is it faster in the later stages? Because as game results are determined, the feasible solution space shrinks. Fewer variables, tighter constraints, faster solving.

Gradient Explosion Problem and Barrier Frank-Wolfe

Standard Frank-Wolfe has a technical issue: when prices approach 0, the gradient of LMSR tends toward negative infinity. This can cause instability in the algorithm.

The solution is **Barrier Frank-Wolfe**: optimize not on the complete polytope M but on a slightly "shrunk" version of M. The shrinkage parameter ε will adaptively decrease with iterations—starting further from the boundary (stable), then gradually approaching the true boundary (precise).

Research shows that in practice, 50 to 150 iterations are sufficient for convergence.

Real Performance

A key finding in the paper [2]:

In the first 16 games of the NCAA tournament, Frank-Wolfe Market Makers (FWMM) performed similarly to simple linear constraint market makers (LCMM)—because the integer programming solver was still too slow.

But after 45 games, the first successful 30-minute projection was completed.

Since then, FWMM has outperformed LCMM in market pricing by 38%.

The turning point was when the result space shrank to a point where integer programming could complete solving within the trading time window.

FWMM is like a student who is still warming up in the first half of the exam, but once they get into the zone, they start to crush it. LCMM is the student who consistently performs well but has a limited ceiling. The key difference is: FWMM has a stronger "weapon" (Bregman projection), but it just needs time to "load" (wait for the solver to finish).

Chapter 4: Execution—Why You Can Still Lose Money Even After Calculating

You detected arbitrage. You used Bregman projection to calculate the optimal trade.

Now you need to execute.

This is where most strategies fail.

Non-Atomic Execution Problem

Polymarket uses a CLOB (Central Limit Order Book) [9]. Unlike decentralized exchanges, trades on a CLOB are executed sequentially—you cannot guarantee that all orders will be filled simultaneously.

Your arbitrage plan:

Buy YES at $0.30. Buy NO at $0.30. Total cost $0.60. Regardless of the outcome, recover $1.00. Profit $0.40.

Reality:

Submit YES order filled at $0.30 ✓

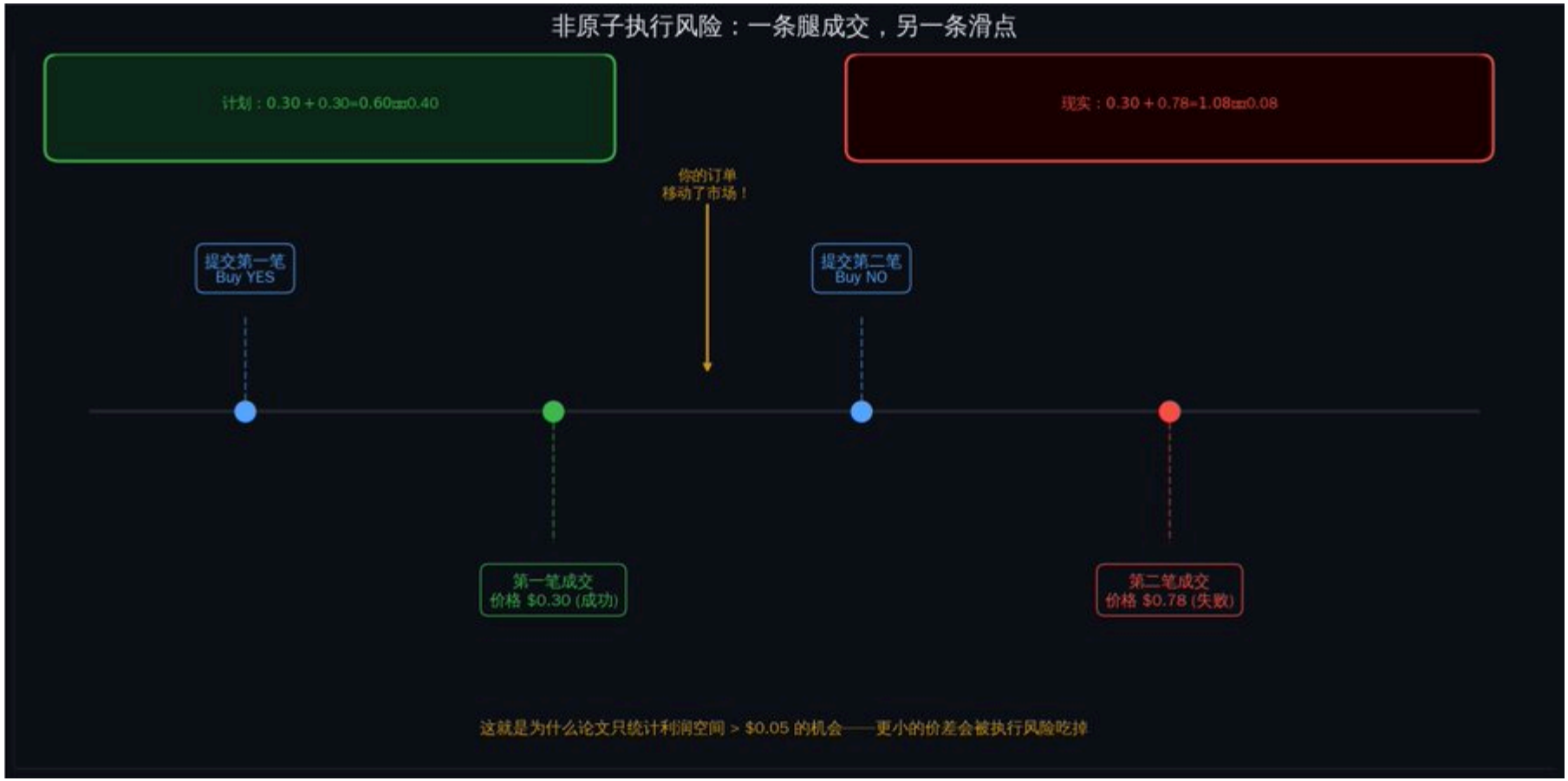
Your order changed the market price.

Submit NO order filled at $0.78 ✗

Total cost: $1.08. Recovery: $1.00. Actual result: lose $0.08.

One leg was filled, the other was not. You are exposed.

This is why the paper only counts opportunities with profit margins exceeding $0.05. Smaller price differences will be eaten up by execution risk.



Non-Atomic Execution Risk

VWAP: The Real Execution Price

Do not assume you can execute at the quoted price. You need to calculate the **Volume Weighted Average Price (VWAP)** [10].

The research team's method is: for each block on the Polygon chain (about every 2 seconds), calculate the VWAP of all YES trades and the VWAP of all NO trades within that block. If `|VWAP_yes + VWAP_no - 1.0| > 0.02`, it is recorded as an arbitrage opportunity [2].

VWAP is "the average price you actually paid." If you want to buy 10,000 tokens, but there are only 2,000 at $0.30, 3,000 at $0.32, and 5,000 at $0.35 on the order book—your VWAP would be $(2000×0.30 + 3000×0.32 + 5000×0.35) / 10000 = 0.326. It’s quite a bit more expensive than the "optimal price" of $0.30.

Liquidity Constraints: How Much You Can Earn Depends on Order Book Depth

Even if prices do deviate, the profit you can earn is limited by available liquidity.

A real example [2]:

Market shows arbitrage: YES price sum = $0.85. Potential profit: $0.15 per dollar. But the order book depth at these prices is only $234. Maximum extractable profit: $234 × 0.15 = $35.10.

For cross-market arbitrage, you need liquidity on all positions simultaneously. The smallest one determines your upper limit.

This is also why, in existing quantitative platforms, demonstrating the impact of order prices on execution prices is crucial.

Chapter 5: Complete System—What Was Actually Deployed

Theory is clean. The production environment is messy.

This is what a truly operational arbitrage system looks like [2].

Data Pipeline

Real-time Data: WebSocket connection to Polymarket's API [9], receiving order book updates (price/quantity changes), trade pushes, market creation/settlement events.

Historical Data: Query contract events through Alchemy Polygon node API—OrderFilled (trade execution), PositionSplit (new token minting), PositionsMerge (token destruction).

The research team analyzed 86 million trades [2]. This scale requires infrastructure that cannot be handled by scripts.

There are also plans to open-source a high-speed trading API. If you plan to use a similar trading model, I hope to provide you with the API experience as soon as possible, and you can DM me anytime.

Dependency Detection Layer

For 305 U.S. election markets, there are 46,360 pairs of possible combinations to check. Manual analysis is impossible.

The research team used a clever method: employing the DeepSeek-R1-Distill-Qwen-32B large language model for initial screening.

Input: Descriptions of the conditions of two markets.

Output: JSON of valid outcome combinations.

Then three layers of validation: Does each market have exactly one condition true? Is the number of valid combinations less than $n \times m$ (indicating a dependency)? Does the dependent subset meet the arbitrage conditions?

Results: 40,057 pairs independent (no arbitrage), 1,576 pairs with dependencies (potential arbitrage), 374 pairs meeting strict conditions, 13 pairs verified manually as exploitable [2].

The LLM's accuracy in complex multi-condition markets is 81.45%. Sufficient for initial screening, but requires manual verification before execution.

Three-Layer Optimization Engine

Layer 1: Simple Linear Constraints (LCMM). Quickly checks basic rules—"the sum of probabilities equals 1," "if A implies B, then P(A) cannot exceed P(B)." Completed in milliseconds, eliminating obvious pricing errors.

Layer 2: Integer Programming Projection (Frank-Wolfe + Gurobi). This is the core. Parameters: Alpha = 0.9 (extract at least 90% of available arbitrage), initial ϵ = 0.1 (10% shrinkage), convergence threshold = 1e-6, time limit = 30 minutes. Typical iteration count: 50-150. Solving time per iteration: 1-30 seconds.

Layer 3: Execution Validation. Before submitting orders, simulate trades on the current order book. Check: Is liquidity sufficient? What is the expected slippage? What is the guaranteed profit after slippage? Does the profit exceed the minimum threshold ($0.05)? Only execute if all pass.

Position Management: Modified Kelly Formula

The standard Kelly formula [11] tells you how much of your capital to allocate to a trade. But in arbitrage scenarios, it needs to include adjustments for execution risk:
f = (b×p - q) / b × √p

Where b is the arbitrage profit percentage, p is the probability of full execution (estimated based on order book depth), and q = 1 - p.

Upper limit: 50% of order book depth. Exceeding this proportion will significantly move the market with your order.

Final Results

From April 2024 to April 2025, total extracted profit:

Single Condition Arbitrage: Low buy both sides $5,899,287 + High sell both sides $4,682,075 = $10,581,362

Market Rebalancing: Low buy all YES $11,092,286 + High sell all YES $612,189 + Buy all NO $17,307,114 = $29,011,589

Cross-Market Combination Arbitrage: $95,634

Total: $39,688,585

The top 10 arbitrators took home $8,127,849 (20.5% of the total). The top arbitrator: $2,009,632, from 4,049 trades, averaging $496[2].

Not a lottery. Not luck. It's systematic execution of mathematical precision.

Final Reality

While traders are still reading "10 Tips for Prediction Markets," what are quantitative systems doing?

They are using integer programming to detect dependencies among 17,218 conditions. They are calculating optimal arbitrage trades using Bregman projection. They are running the Frank-Wolfe algorithm to handle gradient explosions. They are estimating slippage using VWAP and executing orders in parallel. They are systematically extracting $40 million in guaranteed profits.

The gap is not luck. It is mathematical infrastructure.

The paper is public [1]. The algorithms are known. The profits are real.

The question is: before the next $40 million is extracted, can you build it?

Concept Quick Reference

Marginal Polytope: The space composed of all "valid prices." Prices must be within this space to be arbitrage-free. Can be understood as the "legal area of prices."

Integer Programming: Describes valid outcomes using linear constraints, avoiding brute enumeration. Compresses 2^{63} checks into a few constraints [3].

Bregman Divergence / KL Divergence: A method for measuring the "distance" between two probability distributions, more suitable for price/probability scenarios than Euclidean distance. Movements near extreme prices carry higher weight [5][6].

LMSR (Logarithmic Market Scoring Rule): The pricing mechanism used by Polymarket market makers, where prices represent implied probabilities [4].

Frank-Wolfe Algorithm: An iterative optimization algorithm that adds only one new vertex per iteration, avoiding enumeration of exponentially many valid results [7].

Gurobi: The industry-leading integer programming solver, the "guide" for each iteration of Frank-Wolfe [8].

CLOB (Central Limit Order Book): Polymarket's trading matching mechanism, where orders are executed sequentially and atomicity cannot be guaranteed [9].

VWAP (Volume Weighted Average Price): The average price you actually paid, considering order book depth. More realistic than the "optimal quote" [10].

Kelly Formula: Tells you how much of your capital to allocate to a trade, balancing returns and risks [11].

Non-Atomic Execution: The issue where multiple orders cannot be guaranteed to be filled simultaneously. One leg filled, another not filled = exposure risk.

DeepSeek: A large language model used for initial screening of market dependencies, with an accuracy of 81.45%.

References

[1] Original: https://x.com/RohOnChain/status/2017314080395296995

[2] Research paper "Unravelling the Probabilistic Forest: Arbitrage in Prediction Markets": https://arxiv.org/abs/2508.03474

[3] Theoretical foundation paper "Arbitrage-Free Combinatorial Market Making via Integer Programming": https://arxiv.org/abs/1606.02825

[4] Explanation of LMSR Logarithmic Market Scoring Rule: https://www.cultivatelabs.com/crowdsourced-forecasting-guide/how-does-logarithmic-market-scoring-rule-lmsr-work

[5] Introduction to Bregman Divergence: https://mark.reid.name/blog/meet-the-bregman-divergences.html

[6] KL Divergence - Wikipedia: https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence

[7] Frank-Wolfe Algorithm - Wikipedia: https://en.wikipedia.org/wiki/Frank%E2%80%93Wolfe_algorithm

[8] Gurobi Optimizer: https://www.gurobi.com/

[9] Polymarket CLOB API Documentation: https://docs.polymarket.com/

[10] VWAP Explanation - Investopedia: https://www.investopedia.com/terms/v/vwap.asp

[11] Kelly Formula - Investopedia: https://www.investopedia.com/articles/trading/04/091504.asp

[12] Decrypt report "The $40 Million Free Money Glitch": https://decrypt.co/339958/40-million-free-money-glitch-crypto-prediction-markets

(Source)

Related Projects

Polymarket

Latest News

Alchemy's AI-driven AgentCard gains access to Visa payments network

Coindesk Jun 27, 2026 09:47:13

The euro has risen more than 0.5% against the US dollar today, currently reported at 1.1426

ChainCatcher Jun 26, 2026 20:58:42

Data: BTC falls below 59,000 USD

ChainCatcher Jun 26, 2026 20:43:37

Spanish regulator: There will be no delays or exemptions for the MiCA license transition period

ChainCatcher Jun 26, 2026 20:25:10

Australia's ASIC extends the transition period for cryptocurrency licenses to the end of September, expanding the scope of exemptions

ChainCatcher Jun 26, 2026 20:23:45

SecondFi has completed the final snapshot of the assets of affected users, and it is expected that asset returns can begin in about two weeks

ChainCatcher

Jun 26, 2026 20:07:53

Recent Fundraising

More

Orthogonal	$4.3M	Jun 26
Metal	Jun 25	
Suilend	Jun 25	

New Tokens

More

CAP	CAP	Jun 26
Nesa	NES	Jun 24
Arcium	ARX	Jun 22

Latest Updates on 𝕏

More

Solana Followed PUMPCADE	Jun 25
Kyle Samani Followed world	Jun 25
ZachXBT Followed BeInCrypto	Jun 25

ROOTDATA

A Crypto Asset Data Platform, Committed to Making Crypto Investing Easier.

Products

Research

Listing new

Update Information

Contact Us

Contact Support

Business Cooperation 🚀

Career Hiring!

Project Claim

API

Points

FAQs

App

About

About Us

Disclaimer

Terms & Conditions

Privacy Policy

More

User Guide

Media Kit

Subscribe to RootData email notifications to get crypto trends and high-quality insights anytime, anywhere

Enter your email address

Subscribe